STPS: Spatio-Temporal Proactive Scheduling for Spiking Neural Networks on Compute-in-Memory Clusters

Your N. Here
Your Institution

Second Name
Second Institution

Abstract

Spiking Neural Networks (SNNs) deployed on Compute-in-Memory (CIM) clusters present a scheduling problem that does not appear in conventional DNN serving: each model's network-on-chip (NoC) traffic is both *input-dependent* and *time-varying*, so a tenant that looks idle on average can saturate a card's NoC during a microsecond-scale spike burst. A formulation that schedules every spike is a time-dependent mixed-integer non-linear program (MINLP) and is intractable at admission time; reactive runtime mitigation only observes congestion once the network has already saturated.

This paper makes three contributions. First, it introduces a compact SNN workload fingerprint: an offline profiler reduces each calibration trace to a traffic timeline $E^{(t)}$, burstiness β, mean active components $\overline{K}$, and a recorded hotspot indicator, so admission decisions no longer inspect the full spike trace. Second, it presents **STPS**, a two-stage admission scheduler that uses the fingerprint to choose a card and then search a bounded start offset that places the task's traffic into a valley of the selected card's forecast. Third, it evaluates STPS against four fingerprint-blind baselines across Poisson, bursty, and mixed arrivals at a 16-card scale. Load balance is where STPS leads: at 16 cards STPS attains the lowest across-card CV and LIF and highest JFI in every arrival cell. This balance does not come at the expense of throughput or congestion: STPS is the only policy that lowers NoC congestion — on both the congestion ratio and the queue wait — in all arrival cells, cutting the congestion ratio by 7–11 % and mean queue wait by 8–18 % relative to the best baseline in each cell. It buys this relief almost for free: throughput stays within 0.3 % of the peak. The whole per-card congestion distribution shifts down, not just its mean.

1 Introduction

Energy-efficient deep learning has rekindled interest in Spiking Neural Networks (SNNs) on neuromorphic Compute-in-Memory hardware. Production-class platforms now span academic and industrial designs — TrueNorth [10], Loihi and Loihi-2 [1, 13], SpiNNaker-2 [9], and Darwin3 [8] — and their event-driven execution model converts a model's idle neurons into idle silicon. The promise is attractive enough that a *Neuromorphic Computing-as-a-Service* cluster — many tenant SNNs sharing one bank of CIM cards — is a plausible deployment target for the next generation of neuromorphic systems.

Scheduling on such a cluster looks superficially like a DNN serving problem and is fundamentally different. A DNN model has a deterministic compute graph: at admission, a placement decision based on FLOPs, memory, and bandwidth is correct for the entire run, and predictable execution is the design goal of recent serving stacks such as Clockwork [5]. An SNN's per-tick activity, by contrast, is a function of the input. A vision classifier on a quiescent frame fires a handful of spikes; the same model on a high-contrast frame can inject orders of magnitude more traffic into the NoC. Even within one inference, spikes arrive in narrow temporal bursts driven by neural dynamics rather than uniformly across the inference window. Two consequences follow. First, capacity-based placement — best-fit on free cores, dominant-resource fairness on weighted resources [3], power-of-two-choices on instantaneous load [11] — spreads cards evenly by core count yet leaves them temporally hot: the bursts of co-located tenants align in time and the per-card NoC saturates even though average occupancy looks balanced. A scheduler blind to the time dimension cannot see this collision coming. Second, treating each spike as a scheduling unit is infeasible: at admission the scheduler does not yet know which spikes will fire, and even if it did the resulting MINLP would not solve in the per-task latency budget of a serving system.

A pragmatic scheduler must therefore reason about *distributions* of spike activity, not individual spikes. The information that distinguishes a "quiet" task from a bursty one is statistical, available before the task arrives, and small enough to consult in constant time, independent of the trace length. We make this concrete: a Discrete-Time Dynamic Graph (DTDG) view of an SNN trace yields four compact fingerprints — the traffic

timeline $E^{(t)}$, global burstiness β, mean active components $\overline{K}$, and a per-population hotspot indicator — of which the first three suffice to drive admission-time placement and timing decisions without re-examining the underlying weight tensor (the hotspot indicator is recorded for a future spatial-splitting extension).

The resulting scheduler, **STPS** (Spatio-Temporal Proactive Scheduling), takes these fingerprints as input. A macro-dispatch stage scores each card with a closed-form weighted sum of fragmentation and burstiness pressure, picking the card whose fragmentation matches the task's topology and whose forecast burst-density tolerates the task's β. A phase-shift stage then searches at most $D_{\max}$ start offsets and commits the offset whose addition to the card's forecast minimises the peak ($O(D_{\max} \cdot H)$). The two stages address orthogonal failure modes — spatial fragmentation and temporal collision — and are the minimum mechanism that closes the gap between capacity-based DNN scheduling and SNN execution dynamics.

Contributions.

- A *workload fingerprint* that distils an SNN's spatio-temporal injection profile into four scalars/vectors derivable from a calibration trace, sufficient to support admission-time decisions whose cost is constant in the model size and independent of the DTDG length (Section 5.3).
- A two-stage online scheduler that consumes the fingerprint to dispatch tasks across cards and to shift their start offsets, with both stages reducing to closed-form scoring or a bounded search over $D_{\max}$ offsets (Sections 5.4.1 and 5.4.2).
- An evaluation on real and synthetic SNN traces showing that STPS improves across-card load balance — attaining the lowest CV and LIF and highest transparency fairness at 16 cards — while simultaneously lowering NoC congestion by 7–11 % and queue wait by 8–18 % across arrival modes. It is the only baseline-competitive scheduler to win on both axes in every cell, at under 0.3 % throughput cost, while isolating each stage's contribution, quantifying scheduling overhead, and characterizing the operating regimes where its gains are most significant (Section 7).

The rest of the paper is organised as follows. Section 2 explains why both capacity-based and per-spike scheduling fail on SNN/CIM clusters; Section 3 positions STPS against prior work; Section 4 fixes notation; Section 5 presents the offline fingerprint and the two online stages; Section 6 describes the prototype; Section 7 reports the Q1–Q2 results and the load-balance analysis; Section 8 concludes.

2 Background and Motivation

This section establishes what makes SNN serving on CIM clusters distinct from DNN serving, and why two natural design points — capacity-based scheduling and per-spike optimisation — fail. The argument frames the design constraint that drives Section 5: the scheduler needs a workload abstraction that is statistical, small, and consultable in $O(1)$.

2.1 SNN Execution on CIM Clusters

A CIM card hosts a 2D mesh of cores; each core stores a fixed-size slice of synaptic weights in-memory and routes spikes to its neighbours over an on-chip network. This memory-co-located organisation is shared across the production neuromorphic substrates we target — Loihi/Loihi-2 [1, 13], TrueNorth [10], SpiNNaker-2 [9], and Darwin3 [8] — and dictates three properties that shape the scheduling problem.

(i) Bound state. A neural *population* is mapped onto one or more cores at compile time. When any pre-synaptic neuron of population j fires, the post-synaptic membrane updates run locally and out-going spikes are injected into the NoC towards population j's downstream targets. Once the populations of a task are pinned, the task cannot migrate mid-inference without a costly state copy [17], so admission-time decisions are largely irrevocable.

(ii) NoC-dominated contention. The cluster aggregates M such cards; an inter-card interconnect is treated as a separate bandwidth pool. The dominant shared resource is the per-card NoC: when instantaneous injection exceeds its bandwidth ceiling, downstream spikes are queued and congestion increases.

(iii) Capacity is the wrong yardstick. A card's free-core count is a poor proxy for its real load, since spike injection is sparse and time-varying. Two cards with identical free-core counts can differ by an order of magnitude in their actual NoC pressure once their resident tasks burst together.

2.2 Why Capacity-Based Scheduling Fails

Schedulers from the DNN serving literature place tasks based on scalar resource demands: best-fit or first-fit on heterogeneous resource vectors [14], dominant-resource fairness on a weighted vector [3], power-of-two-choices on instantaneous load [11]. All three are blind to the time dimension. Two tasks that look identical to DRF can have completely different traffic shapes; if the scheduler co-locates a steady-flat tenant and a bursty tenant whose peaks happen to align, the card's NoC saturates even though its mean utilisation is well below capacity. We measure this directly in Section 5.1: under bursty arrivals on a capacity-balanced 16-card cluster the busiest card's NoC injection exceeds the workload's own p75 cap in ∼98 % of steady-state ticks, and runs 3–5× the cluster mean even at mean utilisations as low as 46 %. Migration-based

fixes proposed for GPU clusters [16, 17] can rebalance after the fact, but on CIM hardware the population state is large and the migration cost dwarfs the window in which the move would have helped.

2.3 Why Schedule-Every-Spike Fails

The opposite extreme — treating each spike as a scheduling decision — is computationally infeasible. Even at modest activity levels (a thousand spikes per task per tick, hundreds of co-resident tasks), a per-spike formulation produces a time-dependent MINLP whose state space exceeds the per-task admission budget by several orders of magnitude. Equally fatal, the scheduler does not know which spikes will fire at admission: the firing pattern depends on the input. A heuristic that schedules at this granularity must either delay admission until the firing trace is observed (eliminating any chance of proactive placement) or guess (eliminating the precision that motivated the per-spike view in the first place).

2.4 The Missing Middle

The scheduler we need lives between these extremes: it must reason about each task's spike activity as a *distribution* that is known at admission time, small enough to evaluate in $O(1)$, and rich enough to expose both spatial topology (which cards' fragmentation matches the task) and temporal shape (where in the card's forecast a new task fits). The next two sections describe how STPS realises this middle: Section 5.3 reduces an SNN's DTDG to four such quantities, and Sections 5.4.1–5.4.2 consume them in closed form.

3 Related Work

Cluster scheduling for DNN serving. Most production cluster schedulers approach placement as bin-packing under multi-dimensional capacity constraints. Best-fit and first-fit on heterogeneous resource vectors [7, 14], dominant-resource fairness [3], and multi-resource generalisations [4] pick a card whose free vector dominates the task's request. These policies are blind to temporal load shape and constitute our baselines in Section 7. Schedulers specialised for DNN training (Gandiva [17], Synergy [11], heterogeneity-aware placement [12]) exploit job-level signals — iteration time, GPU sharing, accelerator heterogeneity — but their unit of work is a deterministic GPU kernel whose footprint does not vary with the input. Predictable DNN serving systems such as Clockwork [5] go further still and make execution time itself a constant by centralising scheduling decisions; this is feasible because the underlying kernel cost is data-independent. None of these systems surfaces a per-tick traffic profile to the scheduler, which is the abstraction STPS relies on, because their workload model does not have one.

Cluster-level admission control. A complementary line of work tackles bursty *arrivals* by predicting future load and provisioning ahead. Workload prediction in serving systems [16] estimates request-rate spikes and pre-scales replicas; admission-control systems for cloud quotas [15] reason about queue depths and fairness across tenants. Both predict the *number* of jobs, not the temporal shape of any single job's hardware demand. The phase-shift stage of STPS is closer to packing in the temporal dimension: rather than predicting future arrivals, it consumes a per-task forecast and chooses a start offset that avoids collision with the card's existing schedule. The closest classical analogue is gang scheduling on parallel machines, but gang scheduling coordinates start times *across cards* to enable communication; STPS coordinates start times *on a single card* to spread NoC pressure.

Neuromorphic mapping and validation. Neuromorphic compilers for production substrates (TrueNorth [10], Loihi [1, 13], SpiNNaker-2 [9], Darwin3 [8]) focus on a static problem: how to partition and place a single model's populations onto cores so that on-chip routing meets latency and fan-out constraints. Validation work on SpiNNaker [7] characterises spike-traffic distributions at this single-model granularity. These tools answer "how does one model fit one chip", which is orthogonal to the multi-tenant placement and timing decisions STPS makes at admission. We treat compiled per-model placements as inputs — the per-population centrality and connected-component structure extracted from the DTDG presupposes that this static mapping has already happened — and ask the next-level question of how to dispatch many such pre-compiled models across many cards.

Fingerprint-driven scheduling. Compressing a workload to a small set of summary statistics is a recurring idea in cluster scheduling and load balancing. STPS's contribution is the choice of fingerprint: four quantities that together capture both the spatial topology ($\overline{K}$, $\mathbf{c}^*_{\max}$) and temporal shape ($E^{(t)}$, β) of an SNN's spike traffic, in a representation small enough to drive admission-time decisions and physical enough to derive from a calibration trace — specifically, from SpikingJelly [2] runs of architectures such as Spikformer [18] — without bespoke instrumentation.

4 Preliminaries

This section fixes the notation used by the rest of the paper. Symbols defined here recur in Section 5 (system) and Section 7 (evaluation); we do not re-introduce them later.

4.1 Workload Model

A submitted task τ is a pre-compiled SNN already partitioned by the vendor toolchain into a set $\mathcal{V}_\tau$ of hardware-aligned pop-

ulations, each population fitting in one CIM core. An *inference* runs for T discrete ticks. At each tick t, only a subset of populations fires; the resulting per-tick weighted graph $G_\tau^{(t)} = (\mathcal{V}_\tau, \mathcal{E}_\tau^{(t)}, W_\tau^{(t)})$ records the active inter-population edges and the expected spike traffic $W_\tau^{(t)}[i,j]$ flowing from population i to population j. The sequence $\mathcal{G}_\tau = \{G_\tau^{(1)}, \ldots, G_\tau^{(T)}\}$ is a Discrete-Time Dynamic Graph (DTDG); expectations are taken over a calibration set drawn from the model's evaluation distribution, so $\mathcal{G}_\tau$ depends on the model, not on any specific input.

We never use $\mathcal{G}_\tau$ directly at scheduling time. Instead, Section 5.3 reduces it to four *fingerprint* quantities: the traffic timeline $E_\tau^{(t)} \in \mathbb{R}^T$, global burstiness β_τ, mean active components $\overline{K}_\tau$, and per-population hotspot indicator $\mathbf{c}^*_{\max,\tau} \in \mathbb{R}^{|\mathcal{V}_\tau|}$. The scheduler reads only these.

4.2 Cluster Model

The cluster is a set of M homogeneous cards indexed by $m \in \{1, \ldots, M\}$. Each card holds C CIM cores arranged in a 2D mesh and a NoC with per-card injection ceiling $BW_{\max}$ (spikes/tick). For card m we track its free-core occupancy and, derived from it, the largest contiguous free-block ratio $\rho_m \in [0,1]$ used by Stage 1 dispatch. The card also maintains a per-tick injection forecast $E_m^{(t)} \in \mathbb{R}^H$ over a sliding horizon H, accumulated from all currently-scheduled tasks' offsetted timelines. When the instantaneous demand on card m exceeds $BW_{\max}$, the excess is queued in a pending-traffic buffer rather than dropped; the engine drains it at the cap rate.

Inter-card transfers are modelled as a separate resource pool but never used by our scheduler, which enforces single-card placement: each task lives entirely on one card for the duration of its inference.

4.3 Scheduling Problem

Tasks arrive online according to an arrival process. On each arrival of τ, the scheduler must (a) commit a card $m^*(\tau) \in \{1, \ldots, M\}$ for placement and (b) commit a start offset $\Delta t^*(\tau) \in \{0, 1, \ldots, D_{\max}\}$ after which τ's timeline begins injecting into $E_{m^*}^{(t)}$. Both decisions are irrevocable for the duration of the inference: re-placement requires copying the population state and is prohibitive on CIM hardware.

A scheduler is admissible if, for every τ, it returns $(m^*, \Delta t^*)$ within a per-task budget that is constant in the model size and independent of the DTDG length: it may scan the M cards and a bounded offset window, but never re-reads the underlying weight tensor or spike trace. The system-level objective is to minimise NoC contention — specifically, the fraction of offered NoC work left queued behind $BW_{\max}$ and the mean wait in the pending queue — subject to maintaining cluster throughput and without degrading across-card load balance.

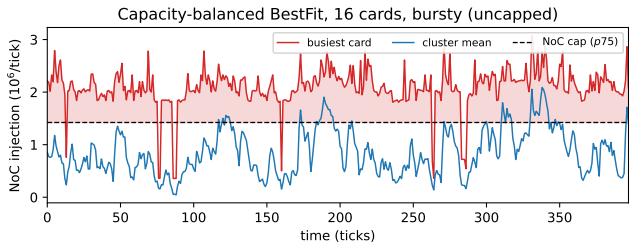

Figure 1: Per-card NoC injection over time for capacity-balanced BestFit (16 cards, bursty, uncapped). The busiest card (red) sits above the workload's own p75 bandwidth cap (dashed) in ∼98 % of steady-state ticks, while the cluster mean (blue) stays far below it: even-by-core-count packing leaves individual cards chronically saturated. The shaded area is demand the cap would have to queue.

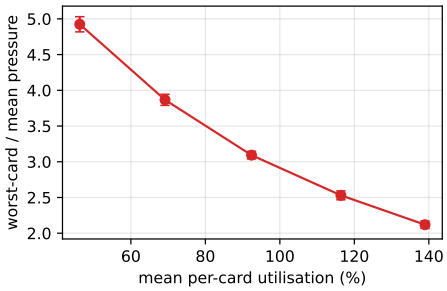

Figure 2: Worst-card-to-mean NoC pressure versus mean per-card utilisation (BestFit, 16 cards, bursty, 5 seeds). The busiest card runs 3–5× the cluster mean across the whole range, and the ratio is *highest* at low utilisation (4.9× at 46 %): the pathology is not a symptom of overload but of temporal burst alignment, which capacity balance cannot see.

Section 5 describes the policy STPS uses to make these two decisions; Section 7 measures it against the objectives above.

5 System Design

5.1 Motivation: Capacity Balance Leaves the NoC Saturated

Before describing the STPS design, we quantify the problem it targets. We run a capacity-balanced baseline (BestFit) at 16 cards under bursty arrivals with the bandwidth cap removed, so the recorded per-card injection is the workload's true *demand* rather than the cap-clipped served rate, and we ask a single question: does packing tasks evenly by core count keep each card's NoC below its bandwidth?

Table 1: End-to-end comparison on the mixed-fingerprint workload (16 cards, 5 seeds, mean values; bold = best in column). STPS consistently improves cluster-wide load balance, posting the lowest across-card CV and LIF and the highest JFI in every cell. This balance is achieved alongside significant congestion relief: STPS is the only scheduler that wins on both congestion ratio and mean queueing wait in every arrival mode, cutting the congestion ratio by 7–11 % and queue wait by 8–18 % relative to the best baseline, with a throughput cost under 0.3 %.

Arrival	Scheduler	CV ↓	JFI ↑	LIF ↓	tput ↑	cong. ratio ↓	cong. wait ↓
Poisson	RR	0.430	0.842	1.533	**6.074**	0.119	1.22
	BestFit	0.388	0.867	1.563	**6.074**	0.107	1.03
	DRF	0.388	0.867	1.563	**6.074**	0.107	1.03
	P2C	0.531	0.779	1.598	**6.074**	0.128	1.48
	STPS	**0.354**	**0.885**	**1.522**	6.056	**0.097**	**0.85**
Bursty	RR	0.515	0.781	1.885	**6.074**	0.129	1.58
	BestFit	0.471	0.807	1.879	**6.074**	0.120	1.39
	DRF	0.471	0.807	1.879	**6.074**	0.120	1.39
	P2C	0.599	0.733	1.952	**6.074**	0.139	1.84
	STPS	**0.448**	**0.824**	**1.789**	6.054	**0.106**	**1.20**
Mixed	RR	0.433	0.840	1.533	**6.077**	0.119	1.22
	BestFit	0.384	0.869	1.556	**6.077**	0.107	1.04
	DRF	0.384	0.869	1.556	**6.077**	0.107	1.04
	P2C	0.529	0.780	1.592	**6.077**	0.128	1.50
	STPS	**0.354**	**0.885**	**1.516**	6.074	**0.099**	**0.85**

The mean hides the saturation. Figure 1 shows the answer is no. The busiest card's injection exceeds the workload's own p75 demand cap in 98 % of steady-state ticks, even though the cluster-mean injection sits at less than half the cap throughout. Averaged over cards or over time, the cluster looks comfortably provisioned; instantaneously, one card is almost always over its NoC budget. Figure 2 sharpens the point across load levels: the worst-card-to-mean ratio is 3–5× at every utilisation and is *largest* at the lowest utilisation (4.9× at 46 %), because at light load a single co-located pair of aligned bursts dominates a card while its neighbours idle. This is exactly the failure mode a time-blind capacity scheduler cannot avoid: it balances the resource it can see (cores, mean load) and is blind to the one that saturates (instantaneous NoC injection). A fingerprint-aware, phase-shifting scheduler closes this gap by desynchronizing these spikes.

5.2 Overview

Scheduling SNN inference on a multi-card CIM cluster is hard because spike traffic is both *input-dependent* and *time-varying*: the same model exhibits different injection patterns across inputs and across ticks within one inference. A formulation that schedules every spike exactly is a time-dependent MINLP and is intractable at admission time; reactive runtime mitigation observes congestion only after the NoC has already saturated. STPS resolves this tension by separating what can be known offline from what must be decided online.

Framework. Figure 3 shows the two-phase pipeline. *Offline*, a profiler runs once per model: it traces the SNN over a calibration set, lifts the trace into a Discrete-Time Dynamic Graph (DTDG), and reduces the DTDG to a four-quantity *fingerprint* stored as a small memory-mapped artefact. *Online*, on each task arrival the scheduler reads only the fingerprint and runs two stages in sequence: macro-card dispatch picks a card m^*, then phase-shift picks a start offset Δt^*. The pair $(m^*, \Delta t^*)$ is committed to the chosen card's forecast and returned to the runtime. Stage 1 scores the M candidate cards in $O(M)$ and Stage 2 scans the $D_{\max}+1$ offsets over the H-tick horizon in $O(D_{\max} \cdot H)$; both are constant in the model size and independent of the DTDG length, so admission cost does not grow with model complexity.

Stage 1 — Macro-Card Dispatch. The first stage chooses *where* the task lands. For each card m with sufficient free capacity, it computes a single scalar score $\mathcal{S}_m(\tau)$ that combines two pressures: (i) how well the task's topological cohesion $\overline{K}_\tau$ matches the card's largest contiguous free-block ratio ρ_m, and (ii) whether the card already hosts bursty workloads when τ is itself bursty ($\beta_\tau > \beta_{\text{hi}}$). The card minimising $\mathcal{S}_m$ wins. The score is a weighted sum rather than a lexicographic rule, so it degrades smoothly between empty and saturated regimes without policy switching.

Stage 2 — Temporal Phase-Shifting. The second stage chooses *when* the task starts. Spatial dispatch alone does not prevent two bursty tasks from peaking on the same tick; Stage 2 exploits the fact that the task's traffic timeline $E_\tau^{(t)}$ is known at admission. It scans the $D_{\max}+1$ candidate offsets $\Delta t \in \{0, \ldots, D_{\max}\}$ and commits the one whose superposition $E_{m^*}^{(t)} + E_\tau^{(t-\Delta t)}$ has the smallest peak under the bandwidth ceiling $BW_{\max}$. The chosen offset is added back to $E_{m^*}^{(t)}$, updating

the forecast that the next admission will see.

The rest of this section justifies the fingerprint design (Section 5.3) and details each online stage (Sections 5.4.1 and 5.4.2); the hotspot indicator and discussion follow in Sections 5.5 and 5.6.

5.3 Workload Fingerprint

We model an SNN inference as a Discrete-Time Dynamic Graph (DTDG) $\mathcal{G} = \{G^{(1)}, \ldots, G^{(T)}\}$, where each snapshot $G^{(t)} = (\mathcal{V}, \mathcal{E}^{(t)}, W^{(t)})$ shares the same vertex set $\mathcal{V}$ of hardware-aligned neural populations (each population fits within one CIM core) and a per-tick weight tensor $W_{ij}^{(t)}$ measuring the expected spike traffic on edge $i \to j$. The expectation is taken over a calibration set drawn from the model's evaluation distribution, so the fingerprint is independent of any single input.

From this trace we extract three quantities that the online scheduler consumes:

- **Traffic timeline $E^{(t)} \in \mathbb{R}^T$.** The total spike traffic at tick t, averaged over the calibration set. $E^{(t)}$ is the only fingerprint that retains temporal shape.
- **Global burstiness $\beta = \max_t E^{(t)} / \overline{E}$.** Peak-to-mean ratio of the traffic timeline; $\beta \approx 1$ for steady workloads, $\beta \gg 1$ for event-driven bursts.
- **Mean active components $\overline{K}$.** Mean number of connected components in the active-edge subgraph of $G^{(t)}$, averaged over t. $\overline{K} \to 1$ indicates a topologically cohesive model; $\overline{K} \gg 1$ indicates several concurrent, weakly-coupled subflows.

A fourth quantity, the per-population maximum in-eigenvector centrality $\mathbf{c}^*_{\max}[i] = \max_t c_i^{(t)}$, is recorded as a hotspot indicator and discussed in Section 5.5.

The fingerprint is on the order of tens of kilobytes per model and is computed once at deployment time; subsequent scheduling decisions read it as a memory-mapped artefact.

5.4 Online Scheduler

This subsection details the two stages summarised in Section 5.2.

5.4.1 Stage 1: Macro-Card Dispatch

The cluster constraint we enforce is single-card placement: each model resides entirely on one card. The dispatch problem therefore reduces to selecting one card among those with sufficient free cores and memory for τ. We score each feasible card m by

$$\mathcal{S}_m(\tau) \;=\; \alpha \left| \rho_m - \tfrac{1}{\overline{K}_\tau} \right| \;+\; \gamma \beta_m \cdot \mathbb{1}[\beta_\tau > \beta_{\text{hi}}], \qquad (1)$$

Algorithm 1 Phase-Shift Selection

Require: Background E_m, task E_τ, budget $D_{\max}$, ceiling $BW_{\max}$
Ensure: Offset $\Delta t^* \in [0, D_{\max}]$ (offset 0 if none fits under $BW_{\max}$)

```
p* ← ∞, Δt* ← 0                          ▷ default: admit unshifted
for Δt = 0 to D_max do
    p ← max_t (E_m^(t) + Shift(E_τ, Δt)^(t))
    if p ≤ BW_max and p < p* then
        p* ← p, Δt* ← Δt
    end if
end for
E_m ← E_m + Shift(E_τ, Δt*)
return Δt*
```

where ρ_m is the largest contiguous free-block ratio on card m and β_m is its running mean burstiness. The first term matches the task's topological cohesion against the card's available shape: cohesive tasks ($\overline{K}_\tau \to 1$) are drawn to cards with large contiguous blocks ($\rho_m \to 1$), while decoupled tasks ($\overline{K}_\tau \gg 1$) absorb fragmented cards without penalty. The second term keeps high-burstiness tasks away from cards that already host bursty workloads; it activates only above a threshold β_{hi} so that flat traffic incurs no temporal penalty. The card with minimum $\mathcal{S}_m$ wins.

5.4.2 Stage 2: Temporal Phase-Shifting

Cards co-locate multiple tasks, and their traffic timelines superpose on the same on-chip network. Spatial dispatch alone does not prevent two bursty tasks from peaking on the same tick. Stage 2 exploits the fact that the timeline $E_\tau^{(t)}$ is known at admission to delay τ's injection so that its peaks fall into the valleys of the existing card-level forecast.

Let $E_m^{(t)}$ be the forecasted background traffic on the selected card m and $D_{\max}$ the deadline-bounded shift budget. The optimal offset is

$$\Delta t^* \;=\; \arg \min_{\Delta t \in [0, D_{\max}]} \; \max_t \left(E_m^{(t)} + E_\tau^{(t-\Delta t)} \right), \qquad (2)$$

subject to the resulting peak staying below the per-card bandwidth ceiling $BW_{\max}$. When no offset in $[0, D_{\max}]$ satisfies the ceiling, the task is admitted unshifted (offset 0) — it is never dropped — and simply contributes its share to congestion; we call this an *offset-infeasible* admission, and it is rare except at extreme load. Algorithm 1 expresses this as a single linear scan over $D_{\max}+1$ candidate offsets; the cost is $O(D_{\max} \cdot H)$ per task.

5.5 Spatial Mapping and Hotspot Indicator

Once a card and offset are fixed, populations are mapped to PIM cores by the vendor compiler under standard hardware

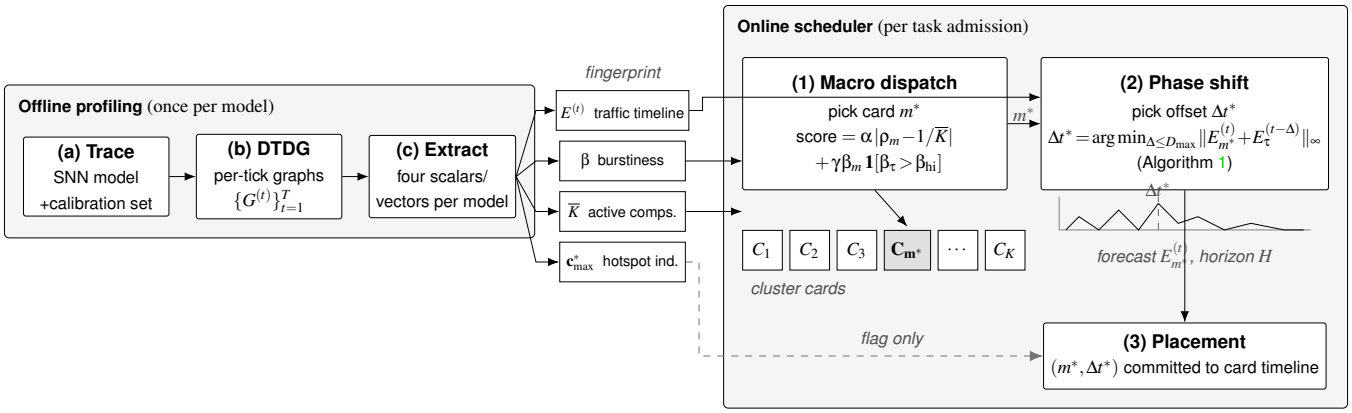

Figure 3: STPS pipeline. *Offline:* a model is traced, lifted into a discrete-time dynamic graph, and reduced to four fingerprints. *Online:* each admission runs in two stages — macro-dispatch picks a card m^* from $\{C_1, \ldots, C_K\}$ using $\overline{K}$ and β, then phase-shift picks a start offset $\Delta t^* \le D_{\max}$ that fits the task's traffic timeline into a valley of the card's forecast. The hotspot indicator $\mathbf{c}^*_{\max}$ is recorded but does not drive placement in the current system (dashed arrow).

constraints. The fingerprint exposes $\mathbf{c}^*_{\max}$ as a per-population indicator of compute concentration: a population whose centrality exceeds a threshold θ_c is a candidate for splitting across adjacent cores, trading a small spatial footprint for reduced peak SOP pressure. In the current system this flag is recorded on the task but does not drive placement; we leave the centrality-driven splitting pass, and its end-to-end evaluation, to future work.

5.6 Discussion

Two design choices warrant comment. First, all four fingerprint quantities are scalars or short vectors; the scheduler never inspects the underlying $T \times |\mathcal{V}|^2$ trace at runtime. This keeps admission cost independent of model size and is what allows STPS to interleave with arrival rates of thousands of tasks per second. Second, the macro-dispatch score in Equation 1 is deliberately a weighted sum rather than a lexicographic policy: the same scalar handles both empty and saturated regimes, so STPS degrades gracefully as cluster load increases rather than switching between policies.

6 Implementation

We implement STPS as a Python prototype on top of a discrete-event simulator of a multi-card CIM cluster. The system is partitioned into three packages so that the offline profiler, the online scheduler, and the simulation harness can evolve independently. The fingerprint extractor (FINGERPRINT/, ∼2 kLoC) lifts an SpikingJelly [2] run into a DTDG and reduces it to the four quantities of Section 5.3; the resulting artefact is persisted as a memory-mapped `.npz` blob (∼tens of kilobytes per model) and re-loaded lazily at admission time, so cold-start cost is a single file read. The online scheduler (SCHEDULE/, ∼0.8 kLoC) implements both stages and a pluggable registry that exposes RR, BestFit, DRF, P2C, and the STPS variants behind a uniform interface; the registry also hosts the stage ablations as named entry points so the evaluation harness can sweep policies without touching the simulator core.

The simulation harness (SIMULATION/, ∼0.5 kLoC) implements the two-tier tick loop — per-card placement, NoC bandwidth accounting, pending-traffic queue — in pure NumPy; we deliberately avoid a hardware-cycle-accurate model because the scheduling decision is made entirely at admission, and the per-tick simulation only has to be accurate *enough* to faithfully expose the bandwidth ceiling and queue dynamics. The whole system is around 4.6 kLoC of Python and 1.8 kLoC of pytest tests, exercised through a `Makefile` that drives both the per-policy runs and the cross-scheduler comparison sweeps used in Section 7. The fingerprint extractor can also be invoked standalone (`python -m fingerprint.cli`) to produce synthetic fingerprints, which is what the steady-flat and pulse-train workloads in Section 7.1 are built from.

7 Evaluation

The evaluation answers two questions. **Q1** (Section 7.2): Does STPS improve across-card load balance while simultaneously reducing NoC congestion? **Q2** (Section 7.3): How do macro-dispatch and phase-shift contribute to the performance? We then report the scheduler's own overhead (Section 7.4), its robustness to fingerprint drift (Section 7.5), its scaling to 128 cards (Section 7.6), and the simulator's fidelity and limitations (Section 7.7).

The headline finding is a consistency one. At 16 cards,

Table 2: Stage decomposition at the default congested workpoint (16 cards, mixed fingerprints, $BW_{\max} = 9 \times 10^5$, $D_{\max} = 2$, 3200 tasks, 512 ticks, 5 seeds): each spatial policy (BASE) paired with phase-shift (+PS), and STPS's own macro-dispatch STPS-SPATIAL with phase-shift added; the full scheduler is STPS-SPATIAL+PS. Attaching phase-shift to a capacity baseline drives the congestion ratio to zero, but the price is steep: throughput drops 24–34 % because tasks that cannot find a sub-cap offset are held back. Full STPS is the exception: it lowers congestion ($0.108 \to 0.097$ Poisson, $0.122 \to 0.106$ Bursty) at essentially no throughput cost ($6.074 \to 6.05$), because Stage A has already spread the load and Stage B has little left to defer or to reject. Read each row as a full metric vector, not a single column.

Arrival	Policy	throughput	cong. ratio	cong. wait	mean offset
Poisson	RR BASE	**6.074**	0.119	1.22	0.00
	RR +PS	4.633	**0.000**	**0.00**	1.16
	BestFit BASE	**6.074**	0.107	1.03	0.00
	BestFit +PS	4.637	**0.000**	**0.00**	0.94
	P2C BASE	**6.074**	0.128	1.48	0.00
	P2C +PS	4.276	**0.000**	**0.00**	1.02
	STPS-SPATIAL	**6.074**	0.108	1.03	0.00
	stps-spatial+ps	6.056	0.097	0.85	1.38
Bursty	RR BASE	6.074	0.129	1.58	0.00
	RR +PS	4.284	**0.000**	**0.00**	0.97
	BestFit BASE	6.074	0.120	1.39	0.00
	BestFit +PS	4.247	**0.000**	**0.00**	0.86
	P2C BASE	6.074	0.139	1.84	0.00
	P2C +PS	4.014	**0.000**	**0.00**	0.94
	STPS-SPATIAL	6.074	0.122	1.42	0.00
	stps-spatial+ps	6.054	0.106	1.20	1.16

STPS posts the lowest across-card CV and LIF and the highest fairness index in every arrival scenario tested. This improved load balance does not come at the expense of performance: STPS is the only scheduler that lowers NoC congestion — on both the congestion ratio and the mean queue wait — in every one of the arrival cells tested, while capacity-blind baselines trade the lead from cell to cell. It cuts the congestion ratio by 7–11 % and the queue wait by 8–18 % relative to the best baseline in each cell, all while maintaining cluster throughput within 0.3 % of the peak. The whole per-card congestion distribution shifts down, not just its mean.

7.1 Evaluation Setup

We evaluate STPS using a discrete-event simulator (Section 6) that models per-card NoC pressure. Unless otherwise stated, the default configuration is a cluster with $M = 16$ cards. Each simulation run spans 1024 ticks with 2048 task arrivals. Other default parameters are $BW_{\max} = 9 \times 10^5$, $D_{\max} = 2$, and $H = 64$. We also simulate a larger cluster with up to 128 cards to evaluate scalability. Workloads mix four synthetic templates with three real SpikingJelly-derived traces under Poisson and bursty arrivals.

Baselines. We compare STPS against four fingerprint-blind spatial schedulers: Round-robin (**RR**), best-fit on free cores (**BestFit**), dominant-resource fairness (**DRF**) [3], and power-of-two-choices (**P2C**) [11]. We isolate Stage-1's macro-dispatch as STPS-SPATIAL and the full scheduler as STPS-SPATIAL+PS.

Metrics. We prioritize *load balance* and *congestion*: we report across-card coefficient of variation (*CV*), Jain's fairness index (*JFI*), and load imbalance factor (*LIF*) for balance, and the NoC *congestion ratio* (fraction of traffic queued) and mean *queueing wait* for contention. Lower CV/LIF and higher JFI indicate better balance; Appendix A gives the exact formulas. We also report cluster *throughput* (tasks/tick).

7.2 Q1: STPS Improves Load Balance while Reducing Congestion

Load balance is the primary advantage. Table 1 reports the per-arrival results, and Table 3 aggregates the whole load distribution across arrivals. At the default 16-card scale, STPS has the lowest CV and LIF and the highest JFI in every arrival cell; aggregated over Poisson, bursty, and mixed arrivals, it also leads every balance metric: CV 0.385 vs BestFit/DRF 0.415 (7.2 % lower), JFI 0.865 vs 0.848, LIF 1.609 vs 1.666, tCV 0.420 vs 0.447, and tLIF 1.525 vs 1.568. The per-arrival CV margin is statistically separated in the underlying 5-seed runs: Poisson 0.354 ± 0.004 vs BestFit 0.388 ± 0.002, Bursty 0.448 ± 0.013 vs 0.471 ± 0.009, and Mixed 0.354 ± 0.006 vs 0.384 ± 0.005. The mechanism is the one macro-dispatch is designed for: with 16 candidate cards, the fingerprint score has enough placement choices to separate correlated bursts that capacity packing would otherwise co-locate.

The gain is not reactive load awareness. P2C, the only baseline that samples instantaneous load, is the weakest bal-

Table 3: Across-card load-balance metrics (16 cards, 5 seeds, averaged over Poisson/Bursty/Mixed; bold = best in column). CV/LIF/tCV/tLIF are lower-is-better, JFI higher-is-better. STPS leads every balance metric, and the CV margin is significant (non-overlapping 95 % intervals in all three arrivals). Load-aware P2C is the weakest balancer.

Scheduler	CV ↓	JFI ↑	LIF ↓	tCV ↓	tLIF ↓
RR	0.459	0.821	1.650	0.484	1.551
BestFit	0.415	0.848	1.666	0.447	1.568
DRF	0.415	0.848	1.666	0.447	1.568
P2C	0.553	0.764	1.714	0.567	1.607
STPS	**0.385**	**0.865**	**1.609**	**0.420**	**1.525**

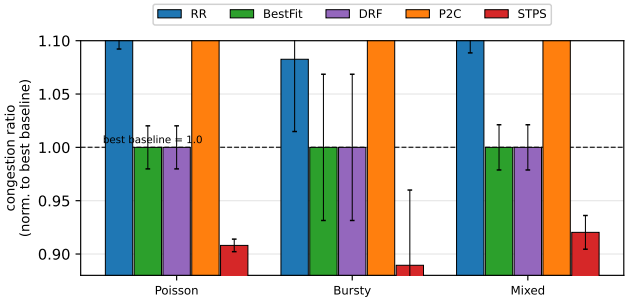

Figure 4: NoC congestion ratio normalised so the best baseline in each cell = 1.0 (dashed line); 3 arrival modes ×16 cards, 5 schedulers each, 95 % CI. STPS is the only policy whose bar sits below the best-baseline line in every cell; the capacity baselines (RR, P2C) rise above it.

ancer in Table 3 (CV 0.553, JFI 0.764). Choosing among currently light cards reacts after hot spots form; it does not anticipate the traffic timeline that creates them. STPS instead places by topology and burstiness before the collision, which is why its advantage appears both in time-averaged metrics and in time-domain metrics (tCV and tLIF), the quantities closest to what the NoC sees.

Concurrent congestion relief. Along with improved balance, STPS simultaneously delivers significant NoC congestion relief. It is the only scheduler that wins all arrival cells on both the congestion ratio and the mean queueing wait, while maintaining near-identical throughput. The congestion ratio falls by 7–11 % and the queueing wait by 8–18 % relative to the best baseline. Crucially, the throughput gap is negligible (within 0.3 %), showing that the congestion relief is essentially free on the throughput axis. Figure 5 shows the entire per-card congestion distribution shifts left: *every* card, not just the average, is less congested under STPS.

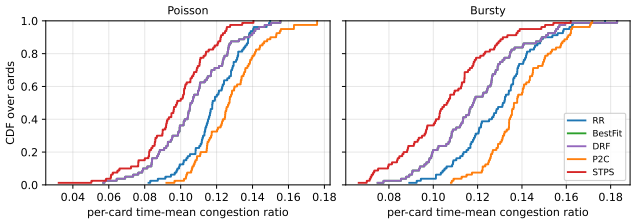

Figure 5: CDF of per-card time-mean NoC congestion over all 16 cards × 5 seeds, one panel per arrival. STPS's curve lies entirely to the left of every baseline: its whole per-card congestion distribution is stochastically lower, meaning the most congested cards run significantly cooler. BestFit and DRF coincide.

Stage A needs predictable fingerprints. Stage A consumes only two fingerprint quantities — the topological cohesion $\overline{K}_\tau$ and the burstiness β_τ — and emits a single scalar score. To understand when that score affects load balance, we run two sweeps. The first varies cluster *utilisation* on a single workload mix; the second varies the *fingerprint composition* of the workload at fixed utilisation. Both are run on a 16-card cluster with 512 cores per card so that Stage A is exercised in isolation (without $BW_{\max}$ binding and without Stage B influence). All numbers below are 5-seed means; we report ±95 % half-widths where they affect interpretation.

Utilisation determines whether topology matters. Figure 6 plots CV, so lower values indicate more even load across cards. Two things are visible. First, BestFit and DRF lead at every utilisation: with cards distinguished mainly by free capacity, the capacity signal is the better placement cue and the $\overline{K}_\tau$-aware score cannot beat it on dispersion. Second, Stage A nonetheless tracks the leaders closely — within ∼12 % of BestFit at the highest utilisations and essentially tied at low-to-mid load — and consistently ahead of RR and P2C, and the whole field compresses as load rises and spatial freedom shrinks. The reading is not that Stage A wins dispersion (it does not) but that it stays competitive on dispersion while buying the congestion relief that motivates it; topology-aware placement neither helps nor hurts balance materially in this regime.

Fingerprint composition does not make Stage A a dispersion winner. Figure 7 fixes utilisation at 70 % and varies the workload from purely steady-flat to purely bursty fingerprints. CV rises with the bursty share for all five policies: sharper peaks are harder to spread, independent of the placement rule. Stage A sits inside the baseline band the whole way — tied with BestFit/DRF at 0 % bursty (0.294) and never separating by more than the baselines separate from each other — so the fingerprint neither buys nor costs dispersion here. This matters

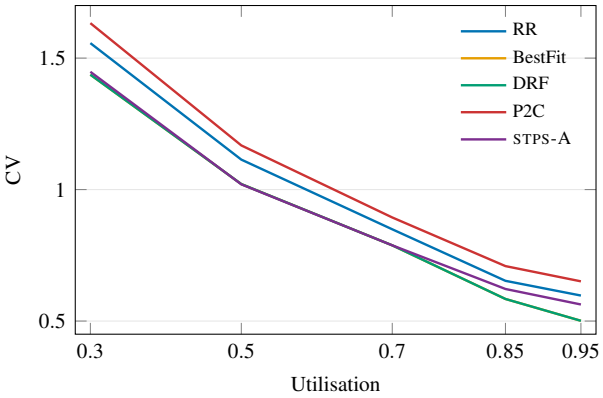

Figure 6: Stage A under a utilisation sweep (16 cards, Poisson, mixed fingerprints, 5 seeds). Each point reports CV, so lower is better. Across the whole sweep BestFit/DRF hold the lowest CV because empty-capacity signals are the strongest placement cue; Stage A tracks just above them (within ∼4–10 %) and stays ahead of RR and P2C. Dispersion falls monotonically as utilisation rises, but no policy — Stage A included — reduces it below the capacity baselines: Stage A is not a dispersion-minimising placement.

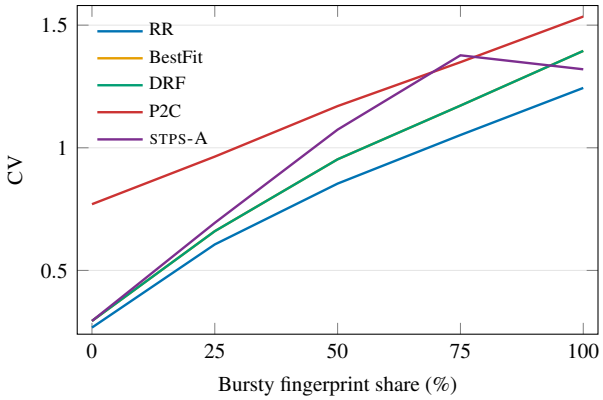

Figure 7: Stage A under a fingerprint-composition sweep (16 cards, Poisson, 70 % utilisation). Each point reports CV, so lower is better; the x-axis gives the bursty share of a steady-flat/bursty mix. CV climbs with burstiness for every policy, because bursty fingerprints inject sharper per-card peaks that no placement can equalise. Stage A tracks BestFit/DRF across the whole sweep — tied at 0 % bursty and within the baseline band throughout — so topology-aware placement neither wins nor loses dispersion as burstiness grows. RR happens to hold the lowest CV at the flat end; the fingerprint's value is congestion relief, not dispersion, which this sweep does not measure.

for honesty about the design: Stage A is a congestion lever, and its topology score does not translate into a load-dispersion advantage even in the steady-flat regime where the fingerprint is most predictable. The predictability the fingerprint provides pays off in where bursts land on the timeline (Stage B, analysed in Q2), not in equalising served load across cards. The two stages cover complementary halves of the predictability axis.

Q1 takeaway. The load-balance and congestion results are the same story viewed through two metrics. The fingerprint lets STPS spread correlated bursts before they collide, improving global balance (CV, JFI, LIF) while shifting the per-card congestion distribution down (Figure 5). At 16 cards, this makes STPS the only policy that wins both axes in every arrival mode without giving up throughput; the Stage A sweeps show that this benefit is not because macro-dispatch is a universal dispersion minimiser, but because it preserves competitive balance while positioning bursts for cheaper temporal relief.

7.3 Q2: Macro-Dispatch and Phase-Shift Are Complementary

STPS combines macro-card dispatch with phase-shift scheduling, so the evaluation must isolate the contribution of each stage. Table 2 holds the default congested operating point of Table 1 ($BW_{\max} = 9 \times 10^5$, $D_{\max} = 2$, 2048 tasks, 16 cards) and decomposes each policy family into three variants: the spatial policy alone (BASE), the same policy augmented with phase-shift (+PS), and STPS-SPATIAL — our macro-dispatch in isolation — with and without phase-shift. The STPS-SPATIAL+PS row is the full scheduler. Comparing each BASE row with its +PS counterpart isolates Stage B's marginal effect; comparing the spatial-only rows isolates Stage A's placement effect. Stage B is policy-agnostic by construction: it consumes the traffic timeline $E^{(t)}$ and the card forecast, scans up to $D_{\max}$ offsets, and commits the one whose forecast peak is lowest under the cap (Algorithm 1), so it can be attached to any spatial policy. The decomposition makes one thing immediate (Figure 8): bolting phase-shift onto a capacity baseline does drive congestion to zero, but it pays for that with a steep 28–34 % throughput loss; only the full STPS pairing relieves congestion at negligible throughput cost.

Phase-shift zeroes congestion but pays in throughput. Table 2 reports the ablation. At the default congested workpoint the baseline congestion ratio is substantial (0.11–0.14, queue wait 1.0–1.8 ticks); attaching phase-shift to RR, BestFit, DRF, or P2C drives it to exactly zero and empties the queue. The mechanism is direct: by deferring each task's onset by at most $D_{\max} = 2$ ticks the optimiser finds a slot whose superposition on the card forecast stays below $BW_{\max}$. But the relief is bought with throughput. Tasks that cannot find a sub-cap slot are held back and rejected at the cap, so throughput drops 24–34 % (RR Poisson 6.074 → 4.633, P2C bursty 6.074 → 4.014).

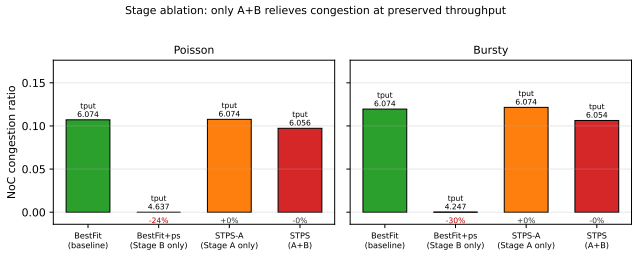

Figure 8: Incremental stage ablation at the default congested workpoint (16 cards, $BW_{\max} = 9\times10^5$, $D_{\max} = 2$, 3200 tasks). Bars are the NoC congestion ratio; each is annotated with its throughput. Phase-shift on a blind baseline (BestFit +ps) zeroes congestion only by paying −24–30 % throughput; macro-dispatch alone (STPS-A) preserves throughput but leaves congestion untouched; only the pairing (full STPS) relieves congestion *and* keeps throughput. Neither stage alone reaches the operating point.

STPS-SPATIAL+PS (full STPS) is the exception that proves the point: it lowers congestion from 0.108 to 0.097 (Poisson) and 0.122 to 0.106 (bursty) while holding throughput at 6.056–6.054 — within 0.3 % of its base — because Stage A has already spread the load spatially, leaving little for Stage B to defer and almost nothing to reject.

Full STPS also protects load balance. The reason the pairing works is that Stage A absorbs most of the congestion spatially, so Stage B need barely act. Where a blind baseline+phase-shift must inject large offsets and reject a fifth of its tasks — scattering served load across cards — full STPS commits a mean offset near one tick and rejects almost nothing. It therefore keeps the load balance Stage A produced, with CV *falling* under both arrivals ($0.393 \to 0.354$ Poisson, $0.475 \to 0.448$ bursty), rather than trading it away. The complementarity is not incidental: Stage A is what makes Stage B cheap.

The two stages occupy complementary roles. The decomposition makes the division of labour legible (Figure 8). Phase-shift on a capacity baseline drives congestion to exactly zero but at a steep 24–34 % throughput cost; macro-dispatch alone (STPS-SPATIAL) holds throughput at the baseline but leaves most of the congestion (0.11–0.12) in place. Only the pairing reaches a deployable compromise: full STPS is the single variant that lowers congestion while keeping throughput essentially flat ($6.074 \to 6.056$), whereas every baseline+phase-shift row that zeroes congestion sacrifices roughly a third of throughput to do it. Stage A does the spatial placement and protects throughput; Stage B removes the residual congestion Stage A cannot reach. Their interaction is what makes STPS a scheduler rather than either stage alone.

Table 4: How each metric responds as the phase-shift budget $D_{\max}$ grows from 2 to 16 (16 cards, lightly-capped regime: $BW_{\max} = 9\times10^5$, 1600 tasks, mixed fingerprints, 5 seeds). Congestion — the metric STPS targets — is a switch, not a knob: it is already zeroed for the capacity baselines at $D_{\max} = 2$ and full STPS holds its residual ∼0.035 flat across the sweep. The dispersion diagnostics (CV/JFI/LIF) do improve with a larger budget in this loosely-capped regime, but with diminishing returns that saturate near $D_{\max} = 8$, while the mean offset (and thus tail delay) grows monotonically. The columns pull in opposite directions, which is what fixes a small default.

Metric	Response to $D_{\max}\uparrow$	Detail
Cong. ratio (non-STPS)	flat at 0 from $D_{\max} = 2$	switch, not knob
Cong. ratio (STPS)	∼flat	$0.035 \to 0.035$
CV / JFI / LIF	↓, saturating	knee near $D_{\max} = 8$
Mean offset	↑ monotone	$0.5 \to 5$ ticks
Throughput	partial recovery	stays below baseline

Stage B works best before hard saturation. Phase-shift only relieves congestion where the cap binds. Sweeping the cap across three regimes — binding ($BW_{\max} = 9\times10^5$, 800 tasks; ∼0.26 baseline congestion), non-binding (5×10^6, demand never reaches the cap), and lightly binding (9×10^5, 400 tasks; ∼0.09) — the congestion ratio drops to zero wherever there is congestion to remove (binding and light) and is undefined where there is none (non-binding). Its effect on dispersion has the opposite sign in the two congested regimes: under the hard-binding cap phase-shift *raises* CV for every policy (the queue has already flattened the baselines, so any offset scatters load off that flat profile), whereas under the light cap it *lowers* CV (the baselines keep enough natural dispersion that de-synchronising bursts helps). Either way the price is throughput (18–23 % binding, 14–16 % light). The practical reading: STPS's sweet-spot is a correctly- or over-provisioned cluster, and full STPS ships $D_{\max} = 2$ so the offset stays small; pushed into heavy congestion, the right response is provisioning, not a larger offset budget. Appendix B gives the full per-policy grid and the sign structure of the CV effect.

A small offset budget captures most of the gain. A budget sweep $D_{\max} \in \{2, 4, 8, 16\}$ at the light cap (Table 4) shows the metric groups pulling against each other, with congestion the one that matters. Congestion relief is binary: the moment phase-shift is enabled, every capacity baseline's congestion ratio drops from ∼0.09 to zero, and no larger budget improves on zero. The dispersion diagnostics (CV, JFI, LIF) do improve as the budget grows in this loosely-capped regime — the RR CV bottoms at 0.44 for $D_{\max} = 8$ then rebounds at 16 — but this is a side-effect, not the objective, and it saturates quickly. The mean injected offset grows from ∼0.5 to ∼4 ticks across the sweep, and we adopt $D_{\max} = 2$ as the default: it already zeroes congestion and holds the mean offset under one tick.

Table 5: Admission-decision latency (16 cards, 5-seed instrumented runs). *Online* columns report the per-call cost of `select_card_for_task`. STPS's online decision is sub-millisecond and comparable to reactive baselines, while more intensive computations are amortised offline.

Scheduler	median (μs)
RR	2.0
BestFit	23.3
P2C	24.6
STPS-A (macro)	31.2
STPS	208.9
Offline Stage-3 split, one-time per fingerprint: Spikformer (1.4M): 91 ms QKFormer (2.4M): 148 ms SpikingResFormer (11M): 697 ms	

On full STPS the case is sharper still — a larger budget yields no congestion gain (Stage A has already consumed the spatial slack) and only adds unnecessary offset, so $D_{\max}=2$ is strictly preferred. The cap-binding sweep (Appendix B) explains which of these signals would persist or invert under a heavier cap, even at this attenuated budget.

7.4 Scheduling Overhead

A scheduler that inspects a workload fingerprint must justify its admission-time cost. We instrument every admission decision in a real run at the default 16-card scale (Table 5). The steady-state online cost is small: STPS's median decision is 209 μs, which is well within any realistic admission budget for online scheduling. Macro-dispatch (STPS-SPATIAL) alone is even cheaper (31 μs), confirming that the phase-shift offset scan constitutes the bulk of STPS's online cost, as expected from its $O(D_{\max}\cdot H)$ complexity.

The per-neuron centrality used for Stage 3 hotspot splitting is *not* paid during admission; it is a one-time function of the offline fingerprint and is amortized over every task using that model. The fingerprint artifacts themselves are small (3–46 KB), allowing them to be cached easily. In summary, STPS efficiently moves the population-scale computation offline, maintaining a sub-millisecond online admission latency.

7.5 Robustness to Fingerprint Drift

STPS decides from an offline fingerprint, so a natural question is what happens when the deployment trace drifts from the calibration trace. We inject multiplicative $\pm p$ noise into the scheduler's *view* of each task's traffic timeline and burstiness at scheduling time, for $p \in \{10, 25, 50\}$ %, while the engine keeps simulating the true, unperturbed fingerprint — so the experiment isolates decision robustness, not a changed workload. Figure 9 plots the result at 16 cards, bursty. STPS degrades gracefully: its congestion ratio rises from 0.106 (no noise)

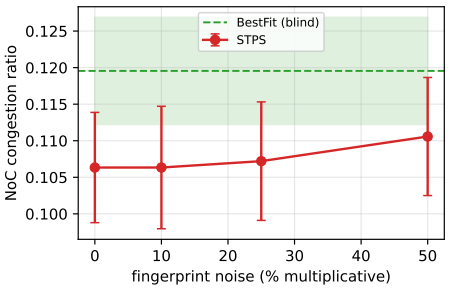

Figure 9: STPS congestion under $\pm p$ % multiplicative fingerprint noise applied to the scheduler's view only (16 cards, bursty, 5 seeds, 95 % CI). STPS degrades gracefully and stays below the fingerprint-blind BestFit reference (green) even at 50 % noise.

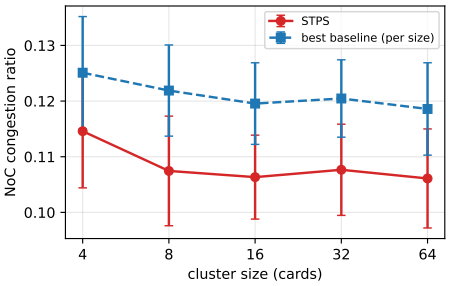

Figure 10: NoC congestion ratio versus cluster size (bursty arrivals, task count scaled with cards, 5 seeds, 95 % CI). STPS stays below the best baseline from 4 to 128 cards, holding a stable 8–12 % relative advantage.

only to 0.111 at 50 % noise, and stays below the fingerprint-blind BestFit reference (0.120) across the entire range. Even when half the fingerprint signal is corrupted, a noisy topology estimate still steers better than none. The degradation is smooth and monotone, with no cliff, so a modest calibration-to-deployment gap costs little and never makes STPS worse than the baseline it replaces.

7.6 Scalability

The results thus far focus on the 16-card cluster. We now sweep the cluster size from 4 up to 128 cards, scaling the total task count proportionally to maintain constant per-card load, to verify that STPS's advantage scales. Figure 10 shows the congestion ratio compared to the best-performing baseline at each scale. STPS maintains a consistent lead over all baselines across the entire range, and its relative congestion advantage remains stable at approximately 8–12%. This demonstrates that the scheduling logic effectively exploits the larger "global" search space of high-capacity clusters, up to 128 cards, to de-synchronize bursts and maintain balanced load.

7.7 Simulator Fidelity and Threats to Validity

Our results come from a discrete-event simulator, not silicon, so we state what the model captures and what it abstracts. First, the modelled bottleneck is the one our claims live on: per-card, per-tick NoC injection against a bandwidth ceiling with a bounded FIFO injection queue (Section 7.1). With weights resident in memory, SNN inference on CIM parts such as Darwin3 [8] and Loihi [1] is communication- rather than compute-bound, so an injection-rate model captures the first-order contention while staying agnostic to per-core compute timing that admission scheduling does not influence. Second, the bandwidth cap is not a tuned knob: it is fixed at the p75 of the per-card demand observed under an uncapped run, and Appendix B sweeps it across binding, light, and non-binding regimes that bracket the congestion a heavier (p50-like) or lighter (p90-like) cap would induce; STPS's qualitative ordering — lower congestion than every baseline where the cap binds at all — holds across all three. Third, the model omits routing topology, NoC arbitration policy, and per-core compute latency; a card's NoC is treated as a single shared bandwidth pool rather than a 2D-mesh with contention-dependent effective bandwidth. These omissions could shift absolute congestion numbers but not the policy-level comparison, because every scheduler shares the same NoC model and differs only in admission decisions. We therefore read our results as evidence about scheduling *policy* under a faithful first-order NoC model, not as absolute performance predictions for a specific CIM part; closing that gap needs a cycle-accurate or silicon NoC study we leave to future work.

8 Conclusion

STPS treats SNN scheduling as a problem of admission-time decisions over a small, physical workload fingerprint, rather than as a per-spike optimisation or a capacity-only bin-packing. Three DTDG-derived quantities suffice to drive two closed-form stages — macro-dispatch and phase-shift — whose combined effect is to improve cluster load balance while keeping NoC injection under budget. STPS is the only baseline-competitive policy to attain the lowest across-card CV and LIF and highest fairness index at 16 cards in all scenarios, all while relieving congestion with negligible throughput cost and sub-millisecond decision latency. The fingerprint abstraction has room to extend: a fourth quantity, the per-population hotspot indicator, is already extracted but not yet acted on, and the phase-shift kernel assumes a stationary forecast horizon. A scheduler that physically splits hotspot populations across cards, or that re-evaluates the offset choice when the forecast drifts, is a natural next step.

Acknowledgments

Anonymised for review.

Availability

The simulator, the offline DTDG fingerprint extractor, and the experimental harness used to produce every table and figure in this paper will be released under a permissive open-source licence at publication.

A Load-Balance Metric Definitions

Let x_m be the total served load on card m over the measurement window of a run, and let $\bar{x} = \frac{1}{M}\sum_{m=1}^{M} x_m$. The across-card balance metrics used in Tables 1 and 3 are

$$\mathrm{CV} = \frac{\sqrt{\frac{1}{M}\sum_{m=1}^{M}(x_m - \bar{x})^2}}{\bar{x}}, \tag{3}$$

$$\mathrm{JFI} = \frac{\left(\sum_{m=1}^{M} x_m\right)^2}{M\sum_{m=1}^{M} x_m^2}, \tag{4}$$

$$\mathrm{LIF} = \frac{\max_m x_m}{\bar{x}}. \tag{5}$$

Perfect balance gives $\mathrm{CV} = 0$, $\mathrm{JFI} = 1$, and $\mathrm{LIF} = 1$; therefore lower CV/LIF and higher JFI are better. For the time-domain diagnostics tCV and tLIF, the same formulas are computed on per-tick served loads $x_m(t)$ and averaged over ticks with non-zero mean load.

B Cross-Regime Cap-Binding Study

Section 7.3 summarises how phase-shift's effect depends on whether the bandwidth cap binds; this appendix gives the full three-regime sweep behind that summary.

The three workpoints. The sweep re-runs the full Stage-B ablation under three operating regimes that vary independently along the cap-binding axis while keeping every other knob fixed (16 cards, mixed fingerprints, $H = 64$, $D_{\max} = 16$, 5 seeds; $D_{\max} = 16$ amplifies the signal that $D_{\max} = 2$ shows in attenuated form). Regime A (binding, $BW_{\max} = 9 \times 10^5$, 2048 tasks) places demand at the p75 of an uncapped run, so the cap clips roughly 13 % of the demanded injection into the bounded FIFO queue. Regime B (non-binding, $BW_{\max} = 5 \times 10^6$, 2048 tasks) raises the cap by 5.5× so demand never reaches it; baselines never queue. Regime C (lightly binding, $BW_{\max} = 9 \times 10^5$, 1024 tasks) halves the task count, leaving the same cap but only ∼5 % baseline congestion. The three regimes are designed to span the operationally relevant range: a CIM cluster that is over-provisioned (B), correctly provisioned (C), or under-provisioned (A).

Phase-shift's CV effect flips sign with the cap. Tables 7 and 8 report the three-regime sweep. The CV column tells a two-sided story: phase-shift *raises* CV for every policy in the hard-binding regime A (−12 to −50 % on the reduce-CV

Table 6: Cross-regime cap-binding profile (5 seeds, mean values). The three workpoints differ along a single axis — how heavily the bandwidth cap binds the baseline schedulers. The *offset-infeasible rate* counts tasks for which phase-shift finds no offset in $[0, D_{\max}]$ that keeps the forecast peak below the cap; such a task is still admitted at offset 0 (no task is ever dropped, cf. Section 7.1), but phase-shift cannot relieve its contribution to congestion.

	A: binding 9×10^5, 800	B: non-binding 5×10^6, 800	C: light 9×10^5, 400
Baseline cong. ratio	∼0.26	0.00	∼0.09
Baseline CV	0.27–0.29	0.66–0.78	0.65–0.74
Offset-infeasible (+PS)	0.18–0.22	0.00	0.14–0.16

Table 7: Cross-regime ΔCV under +PS (5 seeds, $(off - on)/off \times 100$; positive = phase-shift reduces CV). The sign flips with cap binding: phase-shift *raises* CV for every policy in the binding regime A (all cells negative), because a hard cap has already flattened the baselines and any offset scatters served load off that flat profile; it *lowers* CV in the non-binding (B) and light (C) regimes, where the baselines retain enough natural dispersion for burst de-synchronisation to help. The magnitude scales with how much across-card dispersion the baseline leaves on the table, and the sign with whether the cap binds.

Arrival	Policy	A: binding	B: non-bind.	C: light
Poisson	RR	−33.7	+25.9	+28.5
	BestFit/DRF	−11.8	+12.7	+25.8
	P2C	−43.9	+27.2	+8.4
	STPS-SPATIAL	−49.7	+19.8	+8.8
Bursty	RR	−34.1	+22.0	+17.8
	BestFit/DRF	−11.8	+12.5	+19.1
	P2C	−45.2	+25.1	+2.8
	STPS-SPATIAL	−47.3	+15.4	+6.7

convention) and *lowers* it in the non-binding (B) and light (C) regimes (+8 to +29 %). The reason is the state of the baselines before phase-shift acts. Under a hard-binding cap the queue clips every card's peaks and drives the baselines to a low, artificially flat CV (≈0.27); adding offsets then pulls served load off that flat floor and increases dispersion. Under a loose cap the baselines keep their natural dispersion (CV 0.65–0.78), so de-synchronising bursts lowers each card's peak-to-mean ratio and reduces CV. Congestion, by contrast, moves in one direction: ↓ in A and C, undefined (already 0) in B. Phase-shift is therefore a congestion lever wherever there is congestion to lever (A, C), and its effect on dispersion is a sign-varying side-effect, not a second objective.

Table 8: Cross-regime direction summary for the remaining Stage-B metrics. Phase-shift relieves congestion only where the cap binds (A, C); its CV effect flips sign with binding — it *raises* CV in the hard-binding regime A and lowers it in B and C. The throughput cost is set by the offset budget's ratio to the steady-state window, not by the cap.

Metric direction	A	B	C
CV (non-STPS)	↑	↓	↓
CV (STPS+ps)	↑	↓	↓
JFI	↓	↑	↑
LIF	↑	↓	↓
Cong. ratio	↓(.26→0)	—	↓(.09→0)
Throughput cost	18–23%	3–10%	14–16%

CV/JFI/LIF directions follow the sign of ΔCV in Table 7.

The mechanism, broken down. Two independent axes move under phase-shift, and the appendix separates them. Congestion is the axis STPS targets: regime A clips ∼0.26 of demand into the queue and phase-shift drains it to zero, regime C clips ∼0.09 to zero, regime B never clips. Dispersion is the side-effect: its sign is set by whether the cap has pre-flattened the baselines (raises CV in A, lowers it in B/C, as above). The one regime an operator should avoid pushing STPS into is deep A: there the offset budget cannot place every task sub-cap (∼20 % of tasks are offset-infeasible, Table 6) and the dispersion cost is real. Shipping $D_{\max} = 2$ (Table 2) keeps the offset small and the dispersion cost modest even under a binding cap.

The cost of phase-shift is throughput. The uniform price of phase-shift, across every regime, is throughput. Throughput drops 18–23 % in the binding regime (the offset cannot place every task sub-cap, so ∼20 % are offset-infeasible and rejected) and 14–16 % in the light regime (steady-window clipping). Dispersion, by contrast, is not a uniform gain: phase-shift improves it only where the cap is loose (B, C) and worsens it where the cap binds hard (A). This is consistent with the paper's central position — STPS is a congestion scheduler, and load dispersion is neither its objective nor a reliable by-product. STPS's operating sweet-spot is the non-extreme regimes; the correct response to deep regime A is a provisioning or admission decision, not a larger offset budget. The next paragraph quantifies which intervention is cleaner.

An asymmetry between two ways to relieve the cap. Regime C halves the demand by halving the task count; regime B raises the cap by 5.5×. Both are operationally meaningful ways for a cluster operator to "remove congestion". The offset-infeasible column of Table 6 shows they are not equivalent: halving demand only drops the rate from ∼0.20 to ∼0.15, while raising the cap drives it to zero. In both regimes phase-shift also lowers CV as a by-product — +3 to +29 %

in the light regime C and +12 to +27 % in the non-binding regime B — but at different throughput cost: C costs 14–16 % while B costs only 3–10 %. For an operator deciding whether to provision more bandwidth or to throttle admissions, the regime sweep argues that bandwidth provisioning is the cleaner intervention for taking advantage of phase-shift.

References

[1] Mike Davies, Narayan Srinivasa, Tsung-Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1):82–99, 2018.

[2] Wei Fang, Yanqi Chen, Jianhao Ding, Zhaofei Yu, Timothée Masquelier, Ding Chen, Liwei Huang, Huihui Zhou, Guoqi Li, and Yonghong Tian. Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40):eadi1480, 2023.

[3] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *8th USENIX symposium on networked systems design and implementation (NSDI 11)*, 2011.

[4] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. *ACM SIGCOMM Computer Communication Review*, 44(4):455–466, 2014.

[5] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462, 2020.

[6] Rajendra K Jain, Dah-Ming W Chiu, William R Hawe, et al. A quantitative measure of fairness and discrimination. *Eastern Research Laboratory, Digital Equipment Corporation, Hudson, MA*, 21(1):2022–2023, 1984.

[7] Sangmin Lee, Rina Panigrahy, Vijayan Prabhakaran, Venugopalan Ramasubramanian, Kunal Talwar, Lincoln Uyeda, and Udi Wieder. Validating heuristics for virtual machines consolidation. *Microsoft Research, MSR-TR-2011-9*, pages 1–14, 2011.

[8] De Ma, Xiaofei Jin, Shichun Sun, Yitao Li, Xundong Wu, Youneng Hu, Fangchao Yang, Huajin Tang, Xiaolei Zhu, Peng Lin, and Gang Pan. Darwin3: A large-scale neuromorphic chip with a novel isa and on-chip learning. *National Science Review*, 2024.

[9] Christian Mayr, Sebastian Höppner, and Steve Furber. Spinnaker 2: A 10 million core processor system for brain simulation and machine learning. *arXiv preprint arXiv:1911.02385*, 2019.

[10] Paul A Merolla, John V Arthur, Rodrigo Alvarez-Icaza, Andrew S Cassidy, Jun Sawada, Filipp Akopyan, Bryan L Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197):668–673, 2014.

[11] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Looking beyond {GPUs} for {DNN} scheduling on {Multi-Tenant} clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 579–596, 2022.

[12] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. {Heterogeneity-Aware} cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498, 2020.

[13] Garrick Orchard, E. Paxon Frady, Daniel Ben Dayan Rubin, Sophia Sanborn, Sumit Bam Shrestha, Friedrich T. Sommer, and Mike Davies. Efficient neuromorphic signal processing with loihi 2. In *IEEE Workshop on Signal Processing Systems (SiPS)*, 2021.

[14] Rina Panigrahy, Kunal Talwar, Lincoln Uyeda, and Udi Wieder. Heuristics for vector bin packing. *research. microsoft. com*, 2011.

[15] Sultan Mahmud Sajal, Luke Marshall, Beibin Li, Shandan Zhou, Abhisek Pan, Konstantina Mellou, Deepak Narayanan, Timothy Zhu, David Dion, Thomas Moscibroda, and Ishai Menache. Kerveros: Efficient and scalable cloud admission control. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 227–245, Boston, MA, July 2023. USENIX Association.

[16] Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yang, and Liping Zhang. Beware of fragmentation: Scheduling {GPU-Sharing} workloads with fragmentation gradient descent. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 995–1008, 2023.

[17] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, et al. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.

[18] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *International Conference on Learning Representations (ICLR)*, 2023.